
Exercises about query optimizations

Bitmaps – Decoding run-lengths

Transform the following run-length encoding into a matrix of bits where rows correspond to tuples and columns to values. Briefly justify whether it is correct, or not.

BCN	Mad	Gir	Zar
1	1	0	0
0	1	1	0
1	0	1	
0	1	1	
0	0	0	
1	0	1	
0		0	
0		0	

Bitmaps – Decoding run-lengths

Transform the following run-length encoding into a matrix of bits where rows correspond to tuples and columns to values. Briefly justify whether it is correct, or not.

BCN	Mad	Gir	Zar
1	1	0	0
0	1	1	0
1	0	1	
0	1	1	
0	0	0	
1	0	1	
0		0	
0		0	

1. Identify the number of bits used to encode the next run-length

To do so, count the number of bits until you find the first 0

Bitmaps – Decoding run-lengths

Transform the following run-length encoding into a matrix of bits where rows correspond to tuples and columns to values. Briefly justify whether it is correct, or not.

BCN	Mad	Gir	Zar
1	1	0	0
0	1	1	0
1	0	1	
0	1	1	
0	0	0	
1	0	1	
0		0	
0		0	

1. Identify the number of bits used to encode the next run-length

To do so, count the number of bits until you find the first 0

There are 2 bits until finding a 0!

Bitmaps – Decoding run-lengths

Transform the following run-length encoding into a matrix of bits where rows correspond to tuples and columns to values. Briefly justify whether it is correct, or not.

BCN	Mad	Gir	Zar
1	1	0	0
0	1	1	0
1	0	1	
0	1	1	
0	0	0	
1	0	1	
0		0	
0		0	

1. Identify the number of bits used to encode the next run-length

To do so, count the number of bits until you find the first 0

There are 2 bits until finding a 0! Hence, the run-length is encoded using the following 2 bits!

Bitmaps – Decoding run-lengths

Transform the following run-length encoding into a matrix of bits where rows correspond to tuples and columns to values. Briefly justify whether it is correct, or not.

BCN	Mad	Gir	Zar
1	1	0	0
0	1	1	0
1	0	1	
0	1	1	
0	0	0	
1	0	1	
0		0	
0		0	

2. Identify the run-length

Once you know the bits that encode the run-length, pick such bits, and translate them into decimal.

Bitmaps – Decoding run-lengths

Transform the following run-length encoding into a matrix of bits where rows correspond to tuples and columns to values. Briefly justify whether it is correct, or not.

BCN	Mad	Gir	Zar
1	1	0	0
0	1	1	0
1	0	1	
0	1	1	
0	0	0	
1	0	1	
0		0	
0		0	

2. Identify the run-length

Once you know the bits that encode the run-length, pick such bits, and translate them into decimal.

“10” in decimal is 2. Hence, the run-length is 2, which means that the run is composed of 2 consecutive 0s!

Bitmaps – Decoding run-lengths

Transform the following run-length encoding into a matrix of bits where rows correspond to tuples and columns to values. Briefly justify whether it is correct, or not.

BCN	Mad	Gir	Zar
1	1	0	0
0	1	1	0
1	0	1	
0	1	1	
0	0	0	
1	0	1	
0		0	
0		0	

3. Write the identified run-length

BCN	Mad	Gir	Zar
0			
0			
1			

Bitmaps – Decoding run-lengths

Transform the following run-length encoding into a matrix of bits where rows correspond to tuples and columns to values. Briefly justify whether it is correct, or not.

BCN	Mad	Gir	Zar
1	1	0	0
0	1	1	0
1	0	1	
0	1	1	
0	0	0	
1	0	1	
0		0	
0		0	

1. Identify the number of bits used to encode the next run-length

To do so, count the number of bits until you find the first 0

Bitmaps – Decoding run-lengths

Transform the following run-length encoding into a matrix of bits where rows correspond to tuples and columns to values. Briefly justify whether it is correct, or not.

BCN	Mad	Gir	Zar
1	1	0	0
0	1	1	0
1	0	1	
0	1	1	
0	0	0	
1	0	1	
0		0	
0		0	

1. Identify the number of bits used to encode the next run-length

To do so, count the number of bits until you find the first 0

There is 1 bit until finding a 0!

Bitmaps – Decoding run-lengths

Transform the following run-length encoding into a matrix of bits where rows correspond to tuples and columns to values. Briefly justify whether it is correct, or not.

BCN	Mad	Gir	Zar
1	1	0	0
0	1	1	0
1	0	1	
0	1	1	
0	0	0	
1	0	1	
0		0	
0		0	

1. Identify the number of bits used to encode the next run-length

To do so, count the number of bits until you find the first 0

There is 1 bit until finding a 0!
Hence, the run-length is encoded using the following 1 bit!

Bitmaps – Decoding run-lengths

Transform the following run-length encoding into a matrix of bits where rows correspond to tuples and columns to values. Briefly justify whether it is correct, or not.

BCN	Mad	Gir	Zar
1	1	0	0
0	1	1	0
1	0	1	
0	1	1	
0	0	0	
1	0	1	
0		0	
0		0	

2. Identify the run-length

Once you know the bits that encode the run-length, pick such bits, and translate them into decimal.

Bitmaps – Decoding run-lengths

Transform the following run-length encoding into a matrix of bits where rows correspond to tuples and columns to values. Briefly justify whether it is correct, or not.

BCN	Mad	Gir	Zar
1	1	0	0
0	1	1	0
1	0	1	
0	1	1	
0	0	0	
1	0	1	
0		0	
0		0	

2. Identify the run-length

Once you know the bits that encode the run-length, pick such bits, and translate them into decimal.

“1” in decimal is 1. Hence, the run-length is 1, which means that the run is composed of 1 consecutive 0s!

Bitmaps – Decoding run-lengths

Transform the following run-length encoding into a matrix of bits where rows correspond to tuples and columns to values. Briefly justify whether it is correct, or not.

BCN	Mad	Gir	Zar
1	1	0	0
0	1	1	0
1	0	1	
0	1	1	
0	0	0	
1	0	1	
0		0	
0		0	

3. Write the identified run-length

BCN	Mad	Gir	Zar
0			
0			
1			
0			
1			

Bitmaps – Decoding run-lengths

Transform the following run-length encoding into a matrix of bits where rows correspond to tuples and columns to values. Briefly justify whether it is correct, or not.

BCN	Mad	Gir	Zar
1	1	0	0
0	1	1	0
1	0	1	
0	1	1	
0	0	0	
1	0	1	
0		0	
0		0	

1. Identify the number of bits used to encode the next run-length

To do so, count the number of bits until you find the first 0

Bitmaps – Decoding run-lengths

Transform the following run-length encoding into a matrix of bits where rows correspond to tuples and columns to values. Briefly justify whether it is correct, or not.

BCN	Mad	Gir	Zar
1	1	0	0
0	1	1	0
1	0	1	
0	1	1	
0	0	0	
1	0	1	
0		0	
0		0	

1. Identify the number of bits used to encode the next run-length

To do so, count the number of bits until you find the first 0

There is 1 bit until finding a 0!

Bitmaps – Decoding run-lengths

Transform the following run-length encoding into a matrix of bits where rows correspond to tuples and columns to values. Briefly justify whether it is correct, or not.

BCN	Mad	Gir	Zar
1	1	0	0
0	1	1	0
1	0	1	
0	1	1	
0	0	0	
1	0	1	
0		0	
0		0	

1. Identify the number of bits used to encode the next run-length

To do so, count the number of bits until you find the first 0

There is 1 bit until finding a 0!
Hence, the run-length is encoded using the following 1 bit!

Bitmaps – Decoding run-lengths

Transform the following run-length encoding into a matrix of bits where rows correspond to tuples and columns to values. Briefly justify whether it is correct, or not.

BCN	Mad	Gir	Zar
1	1	0	0
0	1	1	0
1	0	1	
0	1	1	
0	0	0	
1	0	1	
0		0	
0		0	

2. Identify the run-length

Once you know the bits that encode the run-length, pick such bits, and translate them into decimal.

Bitmaps – Decoding run-lengths

Transform the following run-length encoding into a matrix of bits where rows correspond to tuples and columns to values. Briefly justify whether it is correct, or not.

BCN	Mad	Gir	Zar
1	1	0	0
0	1	1	0
1	0	1	
0	1	1	
0	0	0	
1	0	1	
0		0	
0		0	

2. Identify the run-length

Once you know the bits that encode the run-length, pick such bits, and translate them into decimal.

“0” in decimal is 0. Hence, the run-length is 0, which means that the run is composed of 0 consecutive 0s!

Bitmaps – Decoding run-lengths

Transform the following run-length encoding into a matrix of bits where rows correspond to tuples and columns to values. Briefly justify whether it is correct, or not.

BCN	Mad	Gir	Zar
1	1	0	0
0	1	1	0
1	0	1	
0	1	1	
0	0	0	
1	0	1	
0		0	
0		0	

3. Write the identified run-length

BCN	Mad	Gir	Zar
0			
0			
1			
0			
1			
1			

Bitmaps – Decoding run-lengths

Transform the following run-length encoding into a matrix of bits where rows correspond to tuples and columns to values. Briefly justify whether it is correct, or not.

BCN	Mad	Gir	Zar
1	1	0	0
0	1	1	0
1	0	1	
0	1	1	
0	0	0	
1	0	1	
0		0	
0		0	

We have finished the encoding of the bit-vector of BCN. So we move to the next bit-vector

Bitmaps – Decoding run-lengths

Transform the following run-length encoding into a matrix of bits where rows correspond to tuples and columns to values. Briefly justify whether it is correct, or not.

BCN	Mad	Gir	Zar
1	1	0	0
0	1	1	0
1	0	1	
0	1	1	
0	0	0	
1	0	1	
0		0	
0		0	

1. Identify the number of bits used to encode the next run-length

To do so, count the number of bits until you find the first 0

Bitmaps – Decoding run-lengths

Transform the following run-length encoding into a matrix of bits where rows correspond to tuples and columns to values. Briefly justify whether it is correct, or not.

BCN	Mad	Gir	Zar
1	1	0	0
0	1	1	0
1	0	1	
0	1	1	
0	0	0	
1	0	1	
0		0	
0		0	

1. Identify the number of bits used to encode the next run-length

To do so, count the number of bits until you find the first 0

There are 3 bit until finding a 0!

Bitmaps – Decoding run-lengths

Transform the following run-length encoding into a matrix of bits where rows correspond to tuples and columns to values. Briefly justify whether it is correct, or not.

BCN	Mad	Gir	Zar
1	1	0	0
0	1	1	0
1	0	1	
0	1	1	
0	0	0	
1	0	1	
0		0	
0		0	

1. Identify the number of bits used to encode the next run-length

To do so, count the number of bits until you find the first 0

There are 3 bit until finding a 0!
Hence, the run-length is encoded using the following 3 bits!

Bitmaps – Decoding run-lengths

Transform the following run-length encoding into a matrix of bits where rows correspond to tuples and columns to values. Briefly justify whether it is correct, or not.

BCN	Mad	Gir	Zar
1	1	0	0
0	1	1	0
1	0	1	
0	1	1	
0	0	0	
1	0	1	
0		0	
0		0	

2. Identify the run-length

Once you know the bits that encode the run-length, pick such bits, and translate them into decimal.

Bitmaps – Decoding run-lengths

Transform the following run-length encoding into a matrix of bits where rows correspond to tuples and columns to values. Briefly justify whether it is correct, or not.

BCN	Mad	Gir	Zar
1	1	0	0
0	1	1	0
1	0	1	
0	1	1	
0	0	0	
1	0	1	
0		0	
0		0	

2. Identify the run-length

Once you know the bits that encode the run-length, pick such bits, and translate them into decimal.

“100” in decimal is 4. Hence, the run-length is 4, which means that the run is composed of 4 consecutive 0s!

Bitmaps – Decoding run-lengths

Transform the following run-length encoding into a matrix of bits where rows correspond to tuples and columns to values. Briefly justify whether it is correct, or not.

BCN	Mad	Gir	Zar
1	1	0	0
0	1	1	0
1	0	1	
0	1	1	
0	0	0	
1	0	1	
0		0	
0		0	

3. Write the identified run-length

BCN	Mad	Gir	Zar
0	0		
0	0		
1	0		
0	0		
1	1		
1			

Bitmaps – Decoding run-lengths

Transform the following run-length encoding into a matrix of bits where rows correspond to tuples and columns to values. Briefly justify whether it is correct, or not.

BCN	Mad	Gir	Zar
1	1	0	0
0	1	1	0
1	0	1	
0	1	1	
0	0	0	
1	0	1	
0		0	
0		0	

1. Identification of the number of bits used to encode the run-lengths
2. Identification of the run-lengths

Bitmaps – Decoding run-lengths

Transform the following run-length encoding into a matrix of bits where rows correspond to tuples and columns to values. Briefly justify whether it is correct, or not.

BCN	Mad	Gir	Zar
1	1	0	0
0	1	1	0
1	0	1	
0	1	1	
0	0	0	
1	0	1	
0		0	
0		0	

Final solution

BCN	Mad	Gir	Zar
0	0	0	1
0	0	1	0
1	0	0	0
0	0	0	0
1	1	0	0
1	0	0	0
0	0	1	0

Bitmaps – Decoding run-lengths

Transform the following run-length encoding into a matrix of bits where rows correspond to tuples and columns to values. Briefly justify whether it is correct, or not.

BCN	Mad	Gir	Zar
1	1	0	0
0	1	1	0
1	0	1	
0	1	1	
0	0	0	
1	0	1	
0		0	
0		0	

Final solution

BCN	Mad	Gir	Zar
0	0	0	1
0	0	1	0
1	0	0	0
0	0	0	0
1	1	0	0
1	0	0	0
0	0	1	0

The bitmap is not OK since the 5th row contains two values (BCN and Mad)

Bitmaps – Encoding run lengths

Bitmaps – Encoding run lengths

ID1	BCN
ID2	Mad
ID3	Gir

1st Block

ID1	Gir
ID2	BCN
ID3	BCN

2nd Block

ID1	Mad
ID2	BCN

3rd Block

ID1	Gir

4th Block

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

Bitmaps – Encoding run lengths

ID1	BCN
ID2	Mad
ID3	Gir

1st Block

ID1	Gir
ID2	BCN
ID3	BCN

2nd Block

ID1	Mad
ID2	BCN

3rd Block

ID1	Gir

4th Block

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

1. Write the uncompressed bitmap

Bitmaps – Encoding run lengths

ID1	BCN
ID2	Mad
ID3	Gir

1st Block

ID1	Gir
ID2	BCN
ID3	BCN

2nd Block

ID1	Mad
ID2	BCN

3rd Block

ID1	Gir

4th Block

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

1. Write the uncompressed bitmap

	BCN	Mad	Gir
Bits of the 1 st block	1	0	0
	0	1	0
	0	0	1
Bits of the 2 nd block	0	0	1
	1	0	0
	1	0	0
Bits of the 3 rd block	0	1	0
	1	0	0
	0	0	0
Bits of the 4 th block	0	0	1
	0	0	0
	0	0	0

Bitmaps – Encoding run lengths

ID1	BCN
ID2	Mad
ID3	Gir

1st Block

ID1	Gir
ID2	BCN
ID3	BCN

2nd Block

ID1	Mad
ID2	BCN

3rd Block

ID1	Gir

4th Block

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

1. Write the uncompressed bitmap

	BCN	Mad	Gir
Bits of the 1 st block	1	0	0
	0	1	0
	0	0	1
Bits of the 2 nd block	0	0	1
	1	0	0
	1	0	0
Bits of the 3 rd block	0	1	0
	1	0	0
	0	0	0
Bits of the 4 th block	0	0	1
	0	0	0
	0	0	0

Take into account that a Hakan factor of 3 means that each block is assumed to contain 3 rows.

If it contains less rows, we Include phantoms in the bitmap

Bitmaps – Encoding run lengths

ID1	BCN
ID2	Mad
ID3	Gir

1st Block

ID1	Gir
ID2	BCN
ID3	BCN

2nd Block

ID1	Mad
ID2	BCN

3rd Block

ID1	Gir

4th Block

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

1. Write the uncompressed bitmap

	BCN	Mad	Gir
Bits of the 1 st block	1	0	0
	0	1	0
	0	0	1
Bits of the 2 nd block	0	0	1
	1	0	0
	1	0	0
Bits of the 3 rd block	0	1	0
	1	0	0
	0	0	0
Bits of the 4 th block	0	0	1
	0	0	0
	0	0	0

Take into account that a Hakan factor of 3 means that each block is assumed to contain 3 rows.

If it contains less rows, we Include phantoms in the bitmap

← Phantom!

← Phantom!
← Phantom!

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

2. Write the half-encoding (encoding in decimal)

BCN	Mad	Gir
1	0	0
0	1	0
0	0	1
0	0	1
1	0	0
1	0	0
0	1	0
1	0	0
0	0	0
0	0	1
0	0	0
0	0	0

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

2. Write the half-encoding (encoding in decimal)

BCN	Mad	Gir
1	0	0
0	1	0
0	0	1
0	0	1
1	0	0
1	0	0
0	1	0
1	0	0
0	0	0
0	0	1
0	0	0
0	0	0

2.1 Look for the first vector of consecutive 0s ending in a 1 (that is, a run)

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

2. Write the half-encoding (encoding in decimal)

2.1 Look for the first vector of consecutive 0s ending in a 1 (that is, a run)

I have found a run composed of zero 0s.!

BCN	Mad	Gir
1	0	0
0	1	0
0	0	1
0	0	1
1	0	0
1	0	0
0	1	0
1	0	0
0	0	0
0	0	1
0	0	0
0	0	0

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

2. Write the half-encoding (encoding in decimal)

BCN	Mad	Gir
1	0	0
0	1	0
0	0	1
0	0	1
1	0	0
1	0	0
0	1	0
1	0	0
0	0	0
0	0	1
0	0	0
0	0	0

2.1 Look for the first vector of consecutive 0s ending in a 1 (that is, a run)

I have found a run composed of zero 0s.!

2.2 Identify the amount of bits you need to encode the length of the run

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

2. Write the half-encoding (encoding in decimal)

BCN	Mad	Gir
1	0	0
0	1	0
0	0	1
0	0	1
1	0	0
1	0	0
0	1	0
1	0	0
0	0	0
0	0	1
0	0	0
0	0	0

2.1 Look for the first vector of consecutive 0s ending in a 1 (that is, a run)

I have found a run composed of zero 0s.!

2.2 Identify the amount of bits you need to encode the length of the run

I need 1 bit to write 0!

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

2. Write the half-encoding (encoding in decimal)

BCN	Mad	Gir
1	0	0
0	1	0
0	0	1
0	0	1
1	0	0
1	0	0
0	1	0
1	0	0
0	0	0
0	0	1
0	0	0
0	0	0

BCN	Mad	Gir
1		
0		

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

2. Write the half-encoding (encoding in decimal)

BCN	Mad	Gir
1	0	0
0	1	0
0	0	1
0	0	1
1	0	0
1	0	0
0	1	0
1	0	0
0	0	0
0	0	1
0	0	0
0	0	0

2.1 Look for the next vector of consecutive 0s ending in a 1 (that is, a run)

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

2. Write the half-encoding (encoding in decimal)

2.1 Look for the next vector of consecutive 0s ending in a 1 (that is, a run)

I have found a run composed of three 0s.!

BCN	Mad	Gir
1	0	0
0	1	0
0	0	1
0	0	1
1	0	0
1	0	0
0	1	0
1	0	0
0	0	0
0	0	1
0	0	0
0	0	0

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

2. Write the half-encoding (encoding in decimal)

BCN	Mad	Gir
1	0	0
0	1	0
0	0	1
0	0	1
1	0	0
1	0	0
0	1	0
1	0	0
0	0	0
0	0	1
0	0	0
0	0	0

2.1 Look for the next vector of consecutive 0s ending in a 1 (that is, a run)

I have found a run composed of three 0s.!

2.2 Identify the amount of bits you need to encode the length of the run

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

2. Write the half-encoding (encoding in decimal)

BCN	Mad	Gir
1	0	0
0	1	0
0	0	1
0	0	1
1	0	0
1	0	0
0	1	0
1	0	0
0	0	0
0	0	1
0	0	0
0	0	0

2.1 Look for the next vector of consecutive 0s ending in a 1 (that is, a run)

I have found a run composed of three 0s.!

2.2 Identify the amount of bits you need to encode the length of the run

I need 2 bits to write 3!

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

2. Write the half-encoding (encoding in decimal)

BCN	Mad	Gir
1	0	0
0	1	0
0	0	1
0	0	1
1	0	0
1	0	0
0	1	0
1	0	0
0	0	0
0	0	1
0	0	0
0	0	0

BCN	Mad	Gir
1		
0		
2		
3		

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

2. Write the half-encoding (encoding in decimal)

BCN	Mad	Gir
1	0	0
0	1	0
0	0	1
0	0	1
1	0	0
1	0	0
0	1	0
1	0	0
0	0	0
0	0	1
0	0	0
0	0	0

2.1 Look for the next vector of consecutive 0s ending in a 1 (that is, a run)

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

2. Write the half-encoding (encoding in decimal)

2.1 Look for the next vector of consecutive 0s ending in a 1 (that is, a run)

I have found a run composed of zero 0s.!

BCN	Mad	Gir
1	0	0
0	1	0
0	0	1
0	0	1
1	0	0
1	0	0
0	1	0
1	0	0
0	0	0
0	0	1
0	0	0
0	0	0

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

2. Write the half-encoding (encoding in decimal)

BCN	Mad	Gir
1	0	0
0	1	0
0	0	1
0	0	1
1	0	0
1	0	0
0	1	0
1	0	0
0	0	0
0	0	1
0	0	0
0	0	0

2.1 Look for the next vector of consecutive 0s ending in a 1 (that is, a run)

I have found a run composed of zero 0s.!

2.2 Identify the amount of bits you need to encode the length of the run

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

2. Write the half-encoding (encoding in decimal)

BCN	Mad	Gir
1	0	0
0	1	0
0	0	1
0	0	1
1	0	0
1	0	0
0	1	0
1	0	0
0	0	0
0	0	1
0	0	0
0	0	0

2.1 Look for the next vector of consecutive 0s ending in a 1 (that is, a run)

I have found a run composed of zero 0s.!

2.2 Identify the amount of bits you need to encode the length of the run

I need 1 bit to write 0!

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

2. Write the half-encoding (encoding in decimal)

BCN	Mad	Gir
1	0	0
0	1	0
0	0	1
0	0	1
1	0	0
1	0	0
0	1	0
1	0	0
0	0	0
0	0	1
0	0	0
0	0	0

BCN	Mad	Gir
1		
0		
2		
3		
1		
0		

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

2. Write the half-encoding (encoding in decimal)

BCN	Mad	Gir
1	0	0
0	1	0
0	0	1
0	0	1
1	0	0
1	0	0
0	1	0
1	0	0
0	0	0
0	0	1
0	0	0
0	0	0

2.1 Look for the next vector of consecutive 0s ending in a 1 (that is, a run)

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

2. Write the half-encoding (encoding in decimal)

2.1 Look for the next vector of consecutive 0s ending in a 1 (that is, a run)

I have found a run composed of one 0s.!

BCN	Mad	Gir
1	0	0
0	1	0
0	0	1
0	0	1
1	0	0
1	0	0
0	1	0
1	0	0
0	0	0
0	0	1
0	0	0
0	0	0

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

2. Write the half-encoding (encoding in decimal)

BCN	Mad	Gir
1	0	0
0	1	0
0	0	1
0	0	1
1	0	0
1	0	0
0	1	0
1	0	0
0	0	0
0	0	1
0	0	0
0	0	0

2.1 Look for the next vector of consecutive 0s ending in a 1 (that is, a run)

I have found a run composed of one 0s.!

2.2 Identify the amount of bits you need to encode the length of the run

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

2. Write the half-encoding (encoding in decimal)

BCN	Mad	Gir
1	0	0
0	1	0
0	0	1
0	0	1
1	0	0
1	0	0
0	1	0
1	0	0
0	0	0
0	0	1
0	0	0
0	0	0

2.1 Look for the next vector of consecutive 0s ending in a 1 (that is, a run)

I have found a run composed of one 0s.!

2.2 Identify the amount of bits you need to encode the length of the run

I need 1 bit to write 1!

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

2. Write the half-encoding (encoding in decimal)

BCN	Mad	Gir
1	0	0
0	1	0
0	0	1
0	0	1
1	0	0
1	0	0
0	1	0
1	0	0
0	0	0
0	0	1
0	0	0
0	0	0

BCN	Mad	Gir
1		
0		
2		
3		
1		
0		
1		
1		

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

Repeat for the rest of runs of the rest of bit vectors

BCN	Mad	Gir
1	0	0
0	1	0
0	0	1
0	0	1
1	0	0
1	0	0
0	1	0
1	0	0
0	0	0
0	0	1
0	0	0
0	0	0

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

Repeat for the rest of runs of the rest of bit vectors

BCN	Mad	Gir
1	0	0
0	1	0
0	0	1
0	0	1
1	0	0
1	0	0
0	1	0
1	0	0
0	0	0
0	0	1
0	0	0
0	0	0

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

Repeat for the rest of runs of the rest of bit vectors

BCN	Mad	Gir
1	0	0
0	1	0
0	0	1
0	0	1
1	0	0
1	0	0
0	1	0
1	0	0
0	0	0
0	0	1
0	0	0
0	0	0

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

Repeat for the rest of runs of the rest of bit vectors

BCN	Mad	Gir
1	0	0
0	1	0
0	0	1
0	0	1
1	0	0
1	0	0
0	1	0
1	0	0
0	0	0
0	0	1
0	0	0
0	0	0

BCN	Mad	Gir
1	1	2
0	1	2
2	3	1
3	4	0
1		3
0		5
1		
1		

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

3. Convert the half-encoding (decimal) into the real encoding (binary)

BCN	Mad	Gir
1	1	2
0	1	2
2	3	1
3	4	0
1		3
0		5
1		
1		

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

3. Convert the half-encoding (decimal) into the real encoding (binary)

3.1 Write a vector of consecutive 1's followed by a 0. The vector should have the number of bits indicated in the half-encoding.

BCN	Mad	Gir
1	1	2
0	1	2
2	3	1
3	4	0
1		3
0		5
1		
1		

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

3. Convert the half-encoding (decimal) into the real encoding (binary)

3.1 Write a vector of consecutive 1's followed by a 0. The vector should have the number of bits indicated in the half-encoding.

I have to write the vector: 0

BCN	Mad	Gir
1	1	2
0	1	2
2	3	1
3	4	0
1		3
0		5
1		
1		

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

3. Convert the half-encoding (decimal) into the real encoding (binary)

3.1 Write a vector of consecutive 1's followed by a 0. The vector should have the number of bits indicated in the half-encoding.

I have to write the vector: 0

3.2 Write the run-length in binary

BCN	Mad	Gir
1	1	2
0	1	2
2	3	1
3	4	0
1		3
0		5
1		
1		

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

3. Convert the half-encoding (decimal) into the real encoding (binary)

3.1 Write a vector of consecutive 1's followed by a 0. The vector should have the number of bits indicated in the half-encoding.

I have to write the vector: 0

3.2 Write the run-length in binary

"0" in binary is: 0

BCN	Mad	Gir
1	1	2
0	1	2
2	3	1
3	4	0
1		3
0		5
1		
1		

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

4. Write the run encoding into the compressed bitmap

BCN	Mad	Gir
1	1	2
0	1	2
2	3	1
3	4	0
1		3
0		5
1		
1		

BCN	Mad	Gir
0		
0		

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

3. Convert the half-encoding (decimal) into the real encoding (binary)

3.1 Write a vector of consecutive 1's followed by a 0. The vector should have the number of bits indicated in the half-encoding.

BCN	Mad	Gir
1	1	2
0	1	2
2	3	1
3	4	0
1		3
0		5
1		
1		

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

3. Convert the half-encoding (decimal) into the real encoding (binary)

3.1 Write a vector of consecutive 1's followed by a 0. The vector should have the number of bits indicated in the half-encoding.

I have to write the vector: 10

BCN	Mad	Gir
1	1	2
0	1	2
2	3	1
3	4	0
1		3
0		5
1		
1		

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

3. Convert the half-encoding (decimal) into the real encoding (binary)

3.1 Write a vector of consecutive 1's followed by a 0. The vector should have the number of bits indicated in the half-encoding.

I have to write the vector: 10

3.2 Write the run-length in binary

"3" in binary is: 11

BCN	Mad	Gir
1	1	2
0	1	2
2	3	1
3	4	0
1		3
0		5
1		
1		

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

4. Write the run encoding into the compressed bitmap

BCN	Mad	Gir
1	1	2
0	1	2
2	3	1
3	4	0
1		3
0		5
1		
1		

BCN	Mad	Gir
0		
0		
1		
0		
1		
1		

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

3. Convert the half-encoding (decimal) into the real encoding (binary)

3.1 Write a vector of consecutive 1's followed by a 0. The vector should have the number of bits indicated in the half-encoding.

BCN	Mad	Gir
1	1	2
0	1	2
2	3	1
3	4	0
1		3
0		5
1		
1		

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

3. Convert the half-encoding (decimal) into the real encoding (binary)

3.1 Write a vector of consecutive 1's followed by a 0. The vector should have the number of bits indicated in the half-encoding.

I have to write the vector: 0

BCN	Mad	Gir
1	1	2
0	1	2
2	3	1
3	4	0
1		3
0		5
1		
1		

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

3. Convert the half-encoding (decimal) into the real encoding (binary)

3.1 Write a vector of consecutive 1's followed by a 0. The vector should have the number of bits indicated in the half-encoding.

I have to write the vector: 0

3.2 Write the run-length in binary

BCN	Mad	Gir
1	1	2
0	1	2
2	3	1
3	4	0
1		3
0		5
1		
1		

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

3. Convert the half-encoding (decimal) into the real encoding (binary)

3.1 Write a vector of consecutive 1's followed by a 0. The vector should have the number of bits indicated in the half-encoding.

I have to write the vector: 0

3.2 Write the run-length in binary

"0" in binary is: 0

BCN	Mad	Gir
1	1	2
0	1	2
2	3	1
3	4	0
1		3
0		5
1		
1		

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

4. Write the run encoding into the compressed bitmap

BCN	Mad	Gir
1	1	2
0	1	2
2	3	1
3	4	0
1		3
0		5
1		
1		

BCN	Mad	Gir
0		
0		
1		
0		
1		
1		
0		
0		

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

3. Convert the half-encoding (decimal) into the real encoding (binary)

3.1 Write a vector of consecutive 1's followed by a 0. The vector should have the number of bits indicated in the half-encoding.

BCN	Mad	Gir
1	1	2
0	1	2
2	3	1
3	4	0
1		3
0		5
1		
1		

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

3. Convert the half-encoding (decimal) into the real encoding (binary)

3.1 Write a vector of consecutive 1's followed by a 0. The vector should have the number of bits indicated in the half-encoding.

I have to write the vector: 0

BCN	Mad	Gir
1	1	2
0	1	2
2	3	1
3	4	0
1		3
0		5
1		
1		

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

3. Convert the half-encoding (decimal) into the real encoding (binary)

3.1 Write a vector of consecutive 1's followed by a 0. The vector should have the number of bits indicated in the half-encoding.

I have to write the vector: 0

3.2 Write the run-length in binary

BCN	Mad	Gir
1	1	2
0	1	2
2	3	1
3	4	0
1		3
0		5
1		
1		

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

3. Convert the half-encoding (decimal) into the real encoding (binary)

3.1 Write a vector of consecutive 1's followed by a 0. The vector should have the number of bits indicated in the half-encoding.

I have to write the vector: 0

3.2 Write the run-length in binary

"1" in binary is: 1

BCN	Mad	Gir
1	1	2
0	1	2
2	3	1
3	4	0
1		3
0		5
1		
1		

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

4. Write the run encoding into the compressed bitmap

BCN	Mad	Gir
1	1	2
0	1	2
2	3	1
3	4	0
1		3
0		5
1		
1		

BCN	Mad	Gir
0		
0		
1		
0		
1		
1		
0		
0		
0		
1		

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

Repeat for the rest of bit-vectors

BCN	Mad	Gir
1	1	2
0	1	2
2	3	1
3	4	0
1		3
0		5
1		
1		

BCN	Mad	Gir
0		
0		
1		
0		
1		
1		
0		
0		
0		
1		

Bitmaps – Encoding run lengths

Assuming a Hakan factor of 3, create the corresponding bitmap using the run-length encoding

Repeat for the rest of bit-vectors

BCN	Mad	Gir
1	1	2
0	1	2
2	3	1
3	4	0
1		3
0		5
1		
1		

BCN	Mad	Gir
0	0	1
0	1	0
1	1	1
0	1	0
1	0	0
1	1	0
0	0	1
0	0	1
0		0
1		1
		0
		1

Exercise about JOINS

Joins

4.2.2 Joins

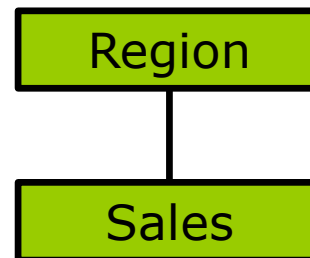
1. Estimate the cost (in terms of disk access) of using bitmap indexes on the foreign keys in the following algorithm implementing a star-join in a RDBMS.

Requirements:

- B-tree associated to each primary key of the dimension tables
- Bitmap on each of the foreign keys of the fact table

Algorithm:

0. Filter each dimension according to the corresponding sub-predicate
 1. Retrieve the bitmaps
 2. Operate (i.e., union, intersection, negation) the corresponding bitmap indexes
 3. Retrieve the required tuples from the fact table
 - a. Complete the tuples with dimensional information
 - i. Use the B-tree associated to the primary key of each dimension



I want the Sales from BCN or Madrid

Notes:

- Step 0 can be saved by defining bitmap join indexes

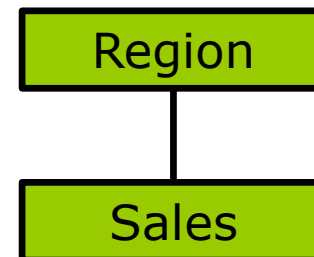
Joins – What is going on?

4.2.2 Joins

1. Estimate the cost (in terms of disk access) of using bitmap indexes on the foreign keys in the following algorithm implementing a star-join in a RDBMS.

Requirements:

- B-tree associated to each primary key of the dimension tables
- Bitmap on each of the foreign keys of the fact table



We have two tables. Sales & Region

I want the Sales from BCN or Madrid

Sales			Region		
RegionID	...	...	ID	Name	...
A			A	BCN	
B			B	Madrid	
C			C	Girona	
...			...	...	

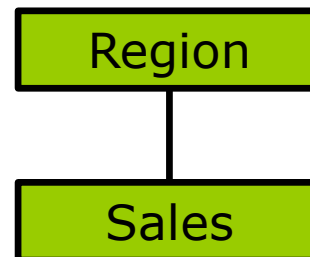
Joins

4.2.2 Joins

1. Estimate the cost (in terms of disk access) of using bitmap indexes on the foreign keys in the following algorithm implementing a star-join in a RDBMS.

Requirements:

- B-tree associated to each primary key of the dimension tables
- Bitmap on each of the foreign keys of the fact table



Sales has a Bitmap index over RegionID

I want the Sales

Sales		
RegionID	...	...
A		
B		
C		
C		
...		

Sales Bitmap		
A	B	C
1	0	0
0	1	0
0	0	1
0	0	1
...		

Madrid

Joins

4.2.2 Joins

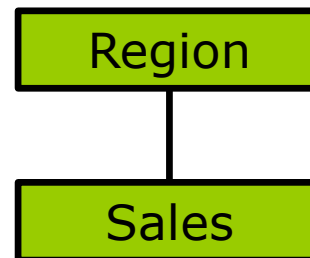
1. Estimate the cost (in terms of disk access) of using bitmap indexes on the foreign keys in the following algorithm implementing a star-join in a RDBMS.

Requirements:

- B-tree associated to each primary key of the dimension tables
- Bitmap on each of the foreign keys of the fact table

If the user requested the Sales where "regionID=A or regionID = B", it would be as easy as performing an OR between bit vectors A and B

Sadly, the user is doing the query with "regionName = 'BCN' or regionName = 'Madrid'" and I do not know which bit vector corresponds to BCN or Madrid...



I want the Sales

Sales Bitmap			Madrid
A	B	C	
1	0	0	
0	1	0	
0	0	1	
0	0	1	
...			

Joins

4.2.2 Joins

1. Estimate the cost (in terms of disk access) of using bitmap indexes on the foreign keys in the following algorithm implementing a star-join in a RDBMS.

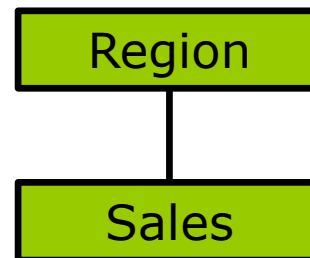
Requirements:

- B-tree associated to each primary key of the dimension tables
- Bitmap on each of the foreign keys of the fact table

Got an idea!

First, I will take a look on the Region table and check which regionID corresponds to BCN, and which regionID corresponds to Madrid.

Then, I will be able to use the bit vectors!



I want the Sales from BCN or Madrid

Region		
ID	Name	...
A	BCN	
B	Madrid	
C	Girona	
...	...	

Joins

4.2.2 Joins

1. Estimate the cost (in terms of disk access) of using bitmap indexes on the foreign keys in the following algorithm implementing a star-join in a RDBMS.

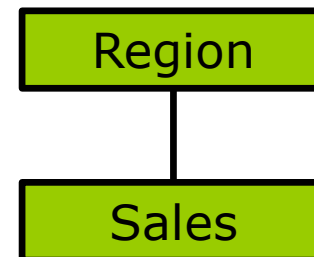
Requirements:

- B-tree associated to each primary key of the dimension tables
- Bitmap on each of the foreign keys of the fact table

Checking the Region table, I see that I need the bit vector corresponding to A, and the bit vector corresponding to B.

Checking the RegionTable is not free, it encompasses performing a FULL SCAN over the region table.

Assuming that Region contains B_{region} blocks, I've read B_{region} blocks from disk



I want the Sales from BCN or Madrid

Region		
ID	Name	...
A	BCN	
B	Madrid	
C	Girona	
...	...	

Joins

4.2.2 Joins

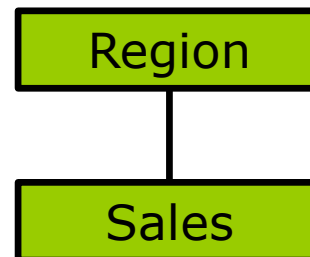
1. Estimate the cost (in terms of disk access) of using bitmap indexes on the foreign keys in the following algorithm implementing a star-join in a RDBMS.

Requirements:

- B-tree associated to each primary key of the dimension tables
- Bitmap on each of the foreign keys of the fact table

Algorithm:

0. Filter each dimension according to the corresponding sub-predicate



I want the Sales from BCN or Madrid

Checking the Region dimension table to get the values (regionIDs) that satisfies the user sub-predicate (being for BCN or Madrid) is the step number 0 of the star-join algorithm.

So, to reiterate, which is the cost of this step?

Joins - Solution

4.2.2 Joins

1. Estimate the cost (in terms of disk access) of using bitmap indexes on the foreign keys in the following algorithm implementing a star-join in a RDBMS.

Requirements:

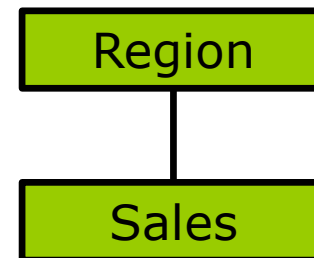
- B-tree associated to each primary key of the dimension tables
- Bitmap on each of the foreign keys of the fact table

Algorithm:

0. Filter each dimension according to the corresponding sub-predicate

Find the Regions of BCN or Madrid

- Full scan the table Region until finding the RegionIDs of BCN and Madrid
- TOTAL: B_{region}



I want the Sales from BCN or Madrid

Joins - Solution

4.2.2 Joins

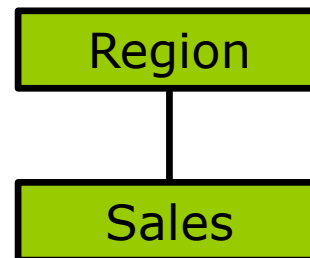
1. Estimate the cost (in terms of disk access) of using bitmap indexes on the foreign keys in the following algorithm implementing a star-join in a RDBMS.

Requirements:

- B-tree associated to each primary key of the dimension tables
- Bitmap on each of the foreign keys of the fact table

Algorithm:

0. Filter each dimension according to the corresponding sub-predicate
1. Retrieve the bitmaps



I want the Sales from BCN or Madrid

In this step we have to retrieve, from disk, the bit vectors of Madrid and BCN (that is, the bit vectors corresponding to regionID=A and regionID=B).

How many blocks are we going to read from disk?

Joins - Solution

4.2.2 Joins

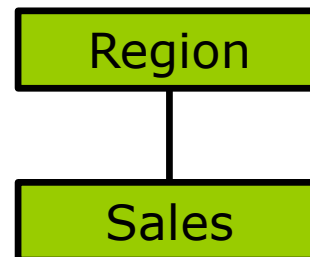
1. Estimate the cost (in terms of disk access) of using bitmap indexes on the foreign keys in the following algorithm implementing a star-join in a RDBMS.

Requirements:

- B-tree associated to each primary key of the dimension tables
- Bitmap on each of the foreign keys of the fact table

Algorithm:

0. Filter each dimension according to the corresponding sub-predicate
1. Retrieve the bitmaps



I want the Sales from BCN or Madrid

Retrieve the bit-vector of BCN, and the bit-vector of Madrid

- Cost of retrieving a bit vector:
 $(\text{Number of bits per vector}) / (\text{Bits per Block})$
- Do note that we have 1 bit for each row in Sales!
- TOTAL: $2 * (\text{Number of Sales}) / (\text{Bits per Block})$

Joins - Solution

4.2.2 Joins

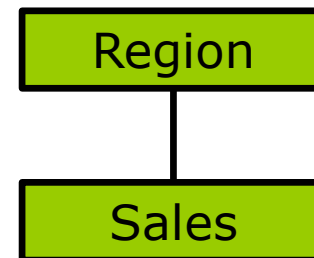
1. Estimate the cost (in terms of disk access) of using bitmap indexes on the foreign keys in the following algorithm implementing a star-join in a RDBMS.

Requirements:

- B-tree associated to each primary key of the dimension tables
- Bitmap on each of the foreign keys of the fact table

Algorithm:

0. Filter each dimension according to the corresponding sub-predicate
 1. Retrieve the bitmaps
 2. Operate (i.e., union, intersection, negation) the corresponding bitmap indexes



*I want the Sales
from BCN or Madrid*

Joins - Solution

4.2.2 Joins

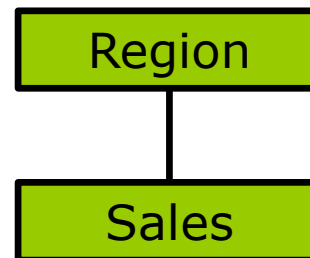
1. Estimate the cost (in terms of disk access) of using bitmap indexes on the foreign keys in the following algorithm implementing a star-join in a RDBMS.

Requirements:

- B-tree associated to each primary key of the dimension tables
- Bitmap on each of the foreign keys of the fact table

Algorithm:

0. Filter each dimension according to the corresponding sub-predicate
 1. Retrieve the bitmaps
 2. Operate (i.e., union, intersection, negation) the corresponding bitmap indexes



I want the Sales from BCN or Madrid

Here we have to apply an OR of the bit-vectors A and B

Joins - Solution

4.2.2 Joins

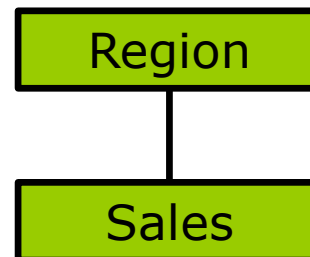
1. Estimate the cost (in terms of disk access) of using bitmap indexes on the foreign keys in the following algorithm implementing a star-join in a RDBMS.

Requirements:

- B-tree associated to each primary key of the dimension tables
- Bitmap on each of the foreign keys of the fact table

Algorithm:

0. Filter each dimension according to the corresponding sub-predicate
 1. Retrieve the bitmaps
 2. Operate (i.e., union, intersection, negation) the corresponding bitmap indexes



I want the Sales from BCN or Madrid

Sales Bitmap		
A	B	A OR B
1	0	1
0	1	1
0	0	0
0	0	0
...		

Joins - Solution

4.2.2 Joins

1. Estimate the cost (in terms of disk access) of using bitmap indexes on the foreign keys in the following algorithm implementing a star-join in a RDBMS.

Requirements:

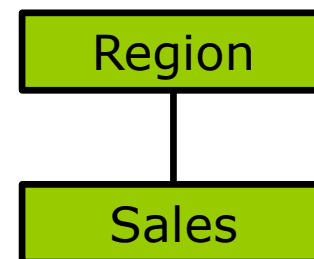
- B-tree associated to each primary key of the dimension tables
- Bitmap on each of the foreign keys of the fact table

Algorithm:

0. Filter each dimension according to the corresponding sub-predicate
 1. Retrieve the bitmaps
 2. Operate (i.e., union, intersection, negation) the corresponding bitmap indexes

Performing bitmap operations does not access the disk!!

The cost of this step is: 0



I want the Sales from BCN or Madrid

Joins - Solution

4.2.2 Joins

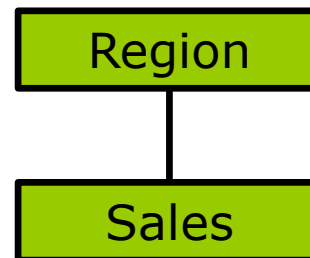
1. Estimate the cost (in terms of disk access) of using bitmap indexes on the foreign keys in the following algorithm implementing a star-join in a RDBMS.

Requirements:

- B-tree associated to each primary key of the dimension tables
- Bitmap on each of the foreign keys of the fact table

Algorithm:

0. Filter each dimension according to the corresponding sub-predicate
 1. Retrieve the bitmaps
 2. Operate (i.e., union, intersection, negation) the corresponding bitmap indexes
 3. Retrieve the required tuples from the fact table



I want the Sales from BCN or Madrid

Untill here we have a bit vector that tells which are the tuples from Sales corresponding to BCN or Madrid. We lack loading them into memory

Joins - Solution

4.2.2 Joins

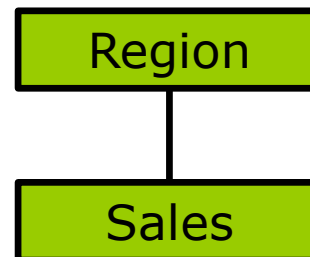
1. Estimate the cost (in terms of disk access) of using bitmap indexes on the foreign keys in the following algorithm implementing a star-join in a RDBMS.

Requirements:

- B-tree associated to each primary key of the dimension tables
- Bitmap on each of the foreign keys of the fact table

Algorithm:

0. Filter each dimension according to the corresponding sub-predicate
 1. Retrieve the bitmaps
 2. Operate (i.e., union, intersection, negation) the corresponding bitmap indexes
 3. Retrieve the required tuples from the fact table



I want the Sales from BCN or Madrid

Sales Bitmap		
A	B	A OR B
1	0	1
0	1	1
0	0	0
0	0	0
...		

Here I see that I have to retrieve tuples 1 and 2, but I do not need to retrieve neither 3, neither 4

Joins - Solution

4.2.2 Joins

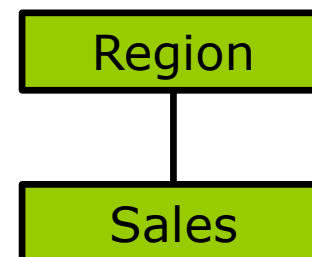
1. Estimate the cost (in terms of disk access) of using bitmap indexes on the foreign keys in the following algorithm implementing a star-join in a RDBMS.

Requirements:

- B-tree associated to each primary key of the dimension tables
- Bitmap on each of the foreign keys of the fact table

Algorithm:

0. Filter each dimension according to the corresponding sub-predicate
 1. Retrieve the bitmaps
 2. Operate (i.e., union, intersection, negation) the corresponding bitmap indexes
 3. Retrieve the required tuples from the fact table



I want the Sales from BCN or Madrid

Sales Bitmap		
A	B	A OR B
1	0	1
0	1	1
0	0	0
0	0	0
...		

Assume that we have 2 rows per block. This means that I need to retrieve the first block, but not the second.

Joins - Solution

4.2.2 Joins

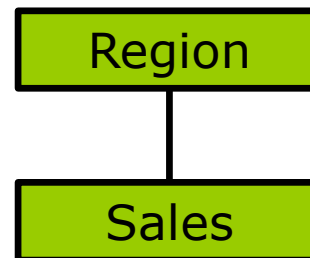
1. Estimate the cost (in terms of disk access) of using bitmap indexes on the foreign keys in the following algorithm implementing a star-join in a RDBMS.

Requirements:

- B-tree associated to each primary key of the dimension tables
- Bitmap on each of the foreign keys of the fact table

Algorithm:

0. Filter each dimension according to the corresponding sub-predicate
 1. Retrieve the bitmaps
 2. Operate (i.e., union, intersection, negation) the corresponding bitmap indexes
 3. Retrieve the required tuples from the fact table



I want the Sales from BCN or Madrid

In essence, in this step I have to count how many blocks do I need to retrieve from Sales. That is, how many blocks contains, at least, one tuple corresponding to Madrid, or BCN.

This is a problem of probabilities!

Joins - Solution

4.2.2 Joins

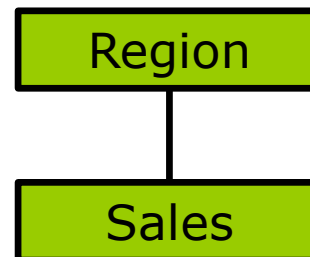
1. Estimate the cost (in terms of disk access) of using bitmap indexes on the foreign keys in the following algorithm implementing a star-join in a RDBMS.

Requirements:

- B-tree associated to each primary key of the dimension tables
- Bitmap on each of the foreign keys of the fact table

Algorithm:

0. Filter each dimension according to the corresponding sub-predicate
 1. Retrieve the bitmaps
 2. Operate (i.e., union, intersection, negation) the corresponding bitmap indexes
 3. Retrieve the required tuples from the fact table



I want the Sales from BCN or Madrid

Pick the blocks of Sales containing some tuple from BCN or Madrid

- Probability of a tuple to be from BCN or Madrid: $\text{Prob}(\text{BCN}) + \text{Prob}(\text{Mad})$
- Probability of a tuple not to be from BCN nor Madrid: $(1 - \text{Prob}(\text{BCN}) - \text{Prob}(\text{Mad}))$
- Probability of R tuples not to be from BCN nor Madrid: $(1 - \text{Prob}(\text{BCN}) - \text{Prob}(\text{Mad}))^R$
- Probability of some of R tuples to be from BCN or Madrid $1 - (1 - \text{Prob}(\text{BCN}) - \text{Prob}(\text{Mad}))^R$
- TOTAL: $B_{\text{sales}} * (1 - (1 - \text{Prob}(\text{BCN}) - \text{Prob}(\text{Mad}))^R)$

R = Rows per block

Joins - Solution

4.2.2 Joins

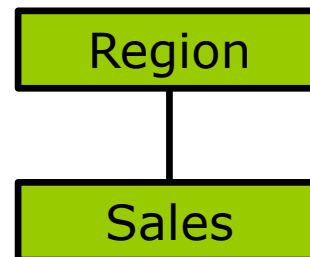
1. Estimate the cost (in terms of disk access) of using bitmap indexes on the foreign keys in the following algorithm implementing a star-join in a RDBMS.

Requirements:

- B-tree associated to each primary key of the dimension tables
- Bitmap on each of the foreign keys of the fact table

Algorithm:

0. Filter each dimension according to the corresponding sub-predicate
 1. Retrieve the bitmaps
 2. Operate (i.e., union, intersection, negation) the corresponding bitmap indexes
 3. Retrieve the required tuples from the fact table
 - a. Complete the tuples with dimensional information
 - i. Use the B-tree associated to the primary key of each dimension



I want the Sales from BCN or Madrid

Joins - Solution

4.2.2 Joins

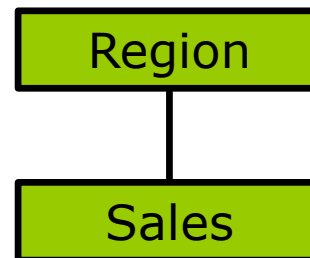
1. Estimate the cost (in terms of disk access) of using bitmap indexes on the foreign keys in the following algorithm implementing a star-join in a RDBMS.

Requirements:

- B-tree associated to each primary key of the dimension tables
- Bitmap on each of the foreign keys of the fact table

Algorithm:

0. Filter each dimension according to the corresponding sub-predicate
 1. Retrieve the bitmaps
 2. Operate (i.e., union, intersection, negation) the corresponding bitmap indexes
 3. Retrieve the required tuples from the fact table
 - a. Complete the tuples with dimensional information
 - i. Use the B-tree associated to the primary key of each dimension



I want the Sales from BCN or Madrid

Until here we have the Sales tuples:

Sales		
RegionID	...	...
A		

Joins - Solution

4.2.2 Joins

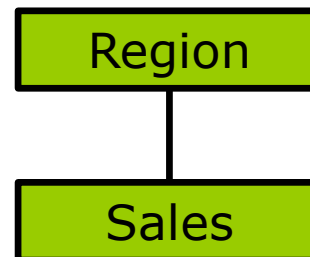
1. Estimate the cost (in terms of disk access) of using bitmap indexes on the foreign keys in the following algorithm implementing a star-join in a RDBMS.

Requirements:

- B-tree associated to each primary key of the dimension tables
- Bitmap on each of the foreign keys of the fact table

Algorithm:

0. Filter each dimension according to the corresponding sub-predicate
 1. Retrieve the bitmaps
 2. Operate (i.e., union, intersection, negation) the corresponding bitmap indexes
 3. Retrieve the required tuples from the fact table
 - a. Complete the tuples with dimensional information
 - i. Use the B-tree associated to the primary key of each dimension



I want the Sales from BCN or Madrid

However, if the user wants to see the regionName, we do not have it!

Sales		
RegionID	...	...
A		

Joins - Solution

4.2.2 Joins

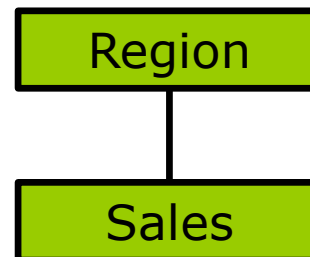
1. Estimate the cost (in terms of disk access) of using bitmap indexes on the foreign keys in the following algorithm implementing a star-join in a RDBMS.

Requirements:

- B-tree associated to each primary key of the dimension tables
- Bitmap on each of the foreign keys of the fact table

Algorithm:

0. Filter each dimension according to the corresponding sub-predicate
 1. Retrieve the bitmaps
 2. Operate (i.e., union, intersection, negation) the corresponding bitmap indexes
 3. Retrieve the required tuples from the fact table
 - a. Complete the tuples with dimensional information
 - i. Use the B-tree associated to the primary key of each dimension



I want the Sales from BCN or Madrid

We need recovering the correspondence:

- regionId A is BCN
- regionId B is Madrid

Joins - Solution

4.2.2 Joins

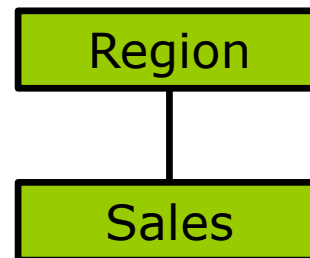
1. Estimate the cost (in terms of disk access) of using bitmap indexes on the foreign keys in the following algorithm implementing a star-join in a RDBMS.

Requirements:

- B-tree associated to each primary key of the dimension tables
- Bitmap on each of the foreign keys of the fact table

Algorithm:

0. Filter each dimension according to the corresponding sub-predicate
 1. Retrieve the bitmaps
 2. Operate (i.e., union, intersection, negation) the corresponding bitmap indexes
 3. Retrieve the required tuples from the fact table
 - a. Complete the tuples with dimensional information
 - i. Use the B-tree associated to the primary key of each dimension



I want the Sales from BCN or Madrid

We need recovering the correspondence:

- regionId A is BCN
- regionId B is Madrid

We do so by accessing a Btree over Region

Joins - Solution

4.2.2 Joins

1. Estimate the cost (in terms of disk access) of using bitmap indexes on the foreign keys in the following algorithm implementing a star-join in a RDBMS.

Requirements:

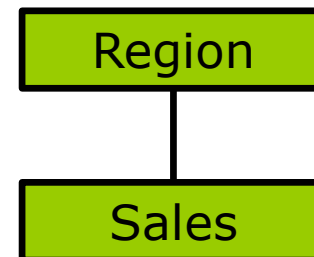
- B-tree associated to each primary key of the dimension tables
- Bitmap on each of the foreign keys of the fact table

Algorithm:

0. Filter each dimension according to the corresponding sub-predicate
 1. Retrieve the bitmaps
 2. Operate (i.e., union, intersection, negation) the corresponding bitmap indexes
 3. Retrieve the required tuples from the fact table
 - a. Complete the tuples with dimensional information
 - i. Use the B-tree associated to the primary key of each dimension

Pick the rows corresponding to Madrid and BCN from Region

- Go through the Btree: h_{tree}
- Go go the data: read 1 block
- TOTAL: $2 * (1 + h_{tree})$



I want the Sales from BCN or Madrid

Joins - Solution

4.2.2 Joins

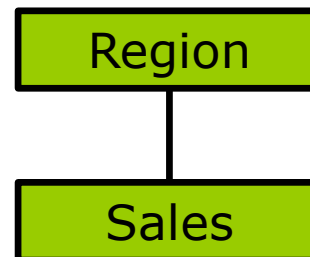
1. Estimate the cost (in terms of disk access) of using bitmap indexes on the foreign keys in the following algorithm implementing a star-join in a RDBMS.

Requirements:

- B-tree associated to each primary key of the dimension tables
- Bitmap on each of the foreign keys of the fact table

Algorithm:

0. Filter each dimension according to the corresponding sub-predicate
 1. Retrieve the bitmaps
 2. Operate (i.e., union, intersection, negation) the corresponding bitmap indexes
 3. Retrieve the required tuples from the fact table
 - a. Complete the tuples with dimensional information
 - i. Use the B-tree associated to the primary key of each dimension



I want the Sales from BCN or Madrid

TOTAL ESTIMATED COST = SUM OF ALL ESTIMATIONS =

$$B_{\text{region}} + 2 * (\text{Number of Sales}) / (\text{Bits per Block}) + 0 + B_{\text{sales}} * (1 - (1 - \text{Prob}(\text{BCN}) - \text{Prob}(\text{Mad}))^R) + 2 * (1 + h_{\text{tree}})$$